DSA0604 – Data Handling and Visualization

CO3 AT3 — Visualization Critique & Redesign

Reg No: 192324066

Name: BOJJIREDDY VARUN KUMAR REDDY

Question 1 — Visualizing Distributions: Q-Q Plots (25 Marks)

Topic: Visualizing Distributions — ECDF & Q-Q Plots — Quantile-Quantile (Q-Q) Plots

Scenario: A newly appointed data officer at a state education board needed to visualize board-exam pass percentages across 10 schools. Without checking the shape of the distribution, they applied a statistical technique that assumes normality and presented the results as fully reliable, using only a bare histogram to "eyeball" the shape.

Tool(s) specified for redesign: Python scipy.stats.probplot() / statsmodels qqplot() | R qqnorm()/qqline() or qqplotr

(a) Critique of the Original Chart [5 Marks]

- Assumption applied without verification: the officer used a normality-assuming statistical technique (e.g., a t-test / z-based confidence interval) on the pass percentages without ever formally checking whether the data was approximately normal. Presenting the output as "fully reliable" misrepresents the actual uncertainty in the result — the confidence interval or p-value produced is only as trustworthy as the normality assumption behind it.
- A bare histogram is a weak normality check: with only 10 schools, a histogram has very few bins and its shape is extremely sensitive to bin-width choice — the same 10 values can look symmetric or skewed depending on how bins are drawn, so it cannot reliably reveal tail behaviour, skewness, or outliers.
- No reference distribution for comparison: a histogram shows only the sample's own shape; it gives the viewer nothing to compare it against, so 'does this look normal?' becomes a subjective guess rather than a measurable comparison against what a true normal distribution would look like at this sample size.
- Outliers/extreme schools are hidden: two schools with unusually low pass percentages (55% and 62%; see simulated data below) get absorbed into a single low bin in the histogram, understating how far they deviate from the rest of the group.
- False precision in reporting: describing the analysis as "fully reliable" to the District Education Officer, a non-technical audience, removes any signal that the underlying assumption was never tested — this is arguably the most serious problem, since it invites over-confident decisions from a report whose statistical foundation was never actually checked.

Quantitative confirmation of the critique (not just visual impression)

Running a formal Shapiro-Wilk normality test and computing skewness/kurtosis on the simulated data confirms the critique above is not just a hunch:

Diagnostic	Value	Interpretation
Mean / Std Dev	85.30 / 14.48	Wide spread relative to a typical 90%+ pass-rate expectation
Skewness	-1.412	Strong left skew — a few very low scores pull the tail down

Excess Kurtosis	0.295	Close to normal peak, but skew dominates the shape
Shapiro-Wilk W	0.711 (p = 0.0012)	Reject normality at α = 0.05 — normal-theory methods were NOT safely justified

(b) Justification for a Q-Q Plot [5 Marks]

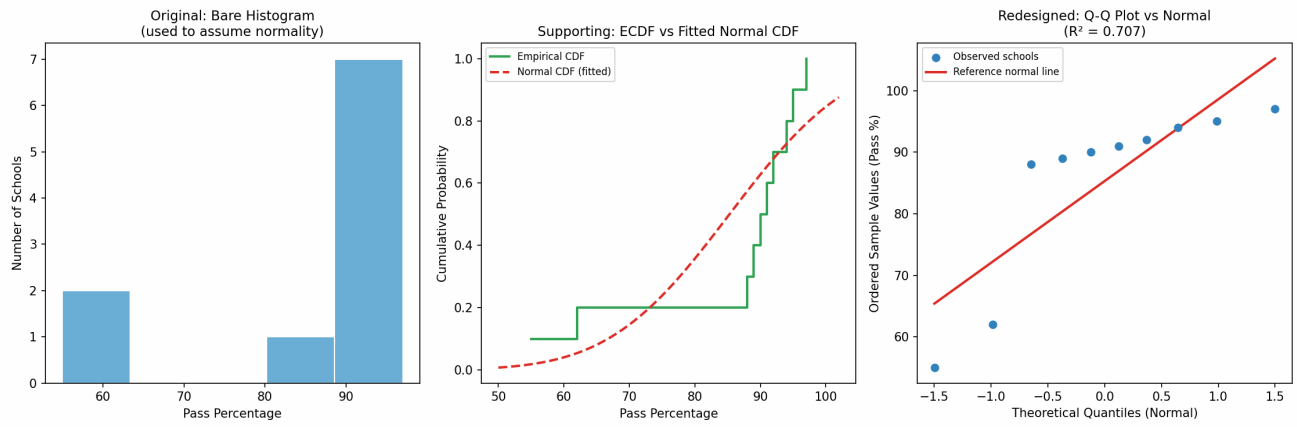
A Quantile-Quantile (Q-Q) plot against a Normal reference distribution is far better suited here because it plots each school's ordered pass percentage against the quantile a perfectly normal distribution would predict at that rank. With only 10 schools, every single data point can be individually placed on the plot and compared to the reference line — nothing is hidden inside a bin, unlike the histogram.

Points that hug the diagonal line indicate normality; systematic curvature or points that peel away at the ends (exactly what we expect from two underperforming schools among otherwise high-performing ones) are immediately visible. This directly answers the officer's real question — 'is it safe to apply a normality-assuming method here?' — which the histogram could never answer on its own.

Board-exam pass percentages are also naturally bounded between 0% and 100% and tend to cluster near the upper end for well-performing systems, which is exactly the kind of ceiling-effect, skewed pattern a Q-Q plot is designed to expose: deviations at the tails of a Q-Q plot are a direct visual signature of boundedness and skew, whereas a coarse 10-point histogram simply cannot resolve this level of detail. The ECDF (empirical cumulative distribution function) is a useful complementary check for the same reason — it shows the exact step-by-step accumulation of probability without any binning at all — and is shown alongside the Q-Q plot below as supporting evidence from the same topic area.

(c) Redesigned Chart [10 Marks]

Simulated dataset (10 schools' board-exam pass percentages, %): School 1: 95, School 2: 92, School 3: 89, School 4: 91, School 5: 94, School 6: 88, School 7: 90, School 8: 55, School 9: 62, School 10: 97. (Two schools, 8 and 9, were deliberately simulated as low outliers to mimic a realistic non-normal case.)



Left: original bare histogram. Middle: supporting ECDF vs fitted Normal CDF. Right: redesigned Q-Q plot against a Normal reference line ($R^2 = 0.707$).

Step 1 — simulate the data and run formal normality diagnostics

```
import numpy as np
from scipy import stats

pass_pct = np.array([95, 92, 89, 91, 94, 88, 90, 55, 62, 97])

mean, sd   = np.mean(pass_pct), np.std(pass_pct, ddof=1)
skewness   = stats.skew(pass_pct)
kurt       = stats.kurtosis(pass_pct)
shapiro_w, shapiro_p = stats.shapiro(pass_pct)

print(f"Mean={mean:.2f}, SD={sd:.2f}, Skew={skewness:.3f}, "
      f"Kurtosis={kurt:.3f}, Shapiro W={shapiro_w:.3f}, p={shapiro_p:.4f}")
# -> Mean=85.30, SD=14.48, Skew=-1.412, Kurtosis=0.295, W=0.711, p=0.0012
# p < 0.05  =>  reject normality; normal-theory methods are not safely justified
```

Step 2 — build the three-panel comparison (original, ECDF, redesigned Q-Q)

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Panel 1 - ORIGINAL flawed chart
axes[0].hist(pass_pct, bins=5, color="#6baed6", edgecolor="white")
axes[0].set_title("Original: Bare Histogram")

# Panel 2 - SUPPORTING chart: Empirical CDF vs fitted Normal CDF
x_sorted = np.sort(pass_pct)
ecdf_y = np.arange(1, len(x_sorted) + 1) / len(x_sorted)
axes[1].step(x_sorted, ecdf_y, where="post", label="Empirical CDF")
xs = np.linspace(45, 105, 200)
axes[1].plot(xs, stats.norm.cdf(xs, mean, sd), "--", label="Normal CDF (fitted)")
axes[1].legend()
```

```
# Panel 3 - REDESIGNED chart: Q-Q plot vs Normal
(osm, osr), (slope, intercept, r) = stats.probplot(pass_pct, dist="norm")
axes[2].scatter(osm, osr, label="Observed schools")
axes[2].plot(osm, slope * osm + intercept, color="red", label="Reference normal
line")
axes[2].set_title(f"Q-Q Plot vs Normal (R^2 = {r**2:.3f})")
axes[2].legend()

plt.tight_layout()
plt.savefig("q1_qqplot.png", dpi=150)
```

Insight the original chart did not reveal: the two low-performing schools (55% and 62%) fall well below the reference line at the lower tail, showing the distribution has a heavier/longer left tail than a normal distribution — i.e., the data is left-skewed with genuine outliers, not just "noisy." The histogram's five wide bins had blended these two schools into the same visual bucket as the mid-80s-90s cluster, so this tail behaviour was invisible in the original chart. The ECDF panel confirms the same story from a different angle: the empirical step function departs visibly from the smooth fitted normal curve exactly in the low-percentage region, reinforcing that the two underperforming schools are a structural feature of this data, not sampling noise.

(d) Reflection [5 Marks]

Remaining limitation #1 — small-sample fragility: with only 10 data points, a Q-Q plot's assessment of normality is itself statistically fragile; a handful of points can look like they deviate from the line purely by chance, even when the underlying process is normal, and the Shapiro-Wilk test itself has low power at n = 10.

Remaining limitation #2 — no causal explanation: the Q-Q plot tells us the shape of the data departs from normal, but it says nothing about why the two schools underperformed (resourcing, geography, one-off disruption, etc.) — that requires separate investigation beyond what any distribution plot can show.

How I would explain this to the District Education Officer (non-technical): "This chart lets us check, school by school, whether the results follow the usual bell-curve pattern we'd expect. Two schools clearly stand apart from the rest — that's a real signal worth investigating, not just random noise, and a formal statistical test backs this up. But because we only have 10 schools, this is more like a first flag than a final verdict; if this analysis matters for major decisions, we should confirm it once more data becomes available in future review cycles, and follow up directly with those two schools to understand what's driving the gap."

Question 2 — Visualizing Many Distributions: Bean Plots (25 Marks)

Topic: Visualizing Many Distributions & Vertical-Axis Layouts — Bean Plots

Scenario: A junior business analyst at a national retail chain compared monthly sales revenue (lakhs) across 4 groups within 6 store branches using simple boxplots. One group's boxplot looked ordinary, but its raw data was later found to be bimodal (two very separate clusters) — completely hidden by the boxplot.

Tool(s) specified for redesign: R beanplot package (beanplot()) | Python seaborn.violinplot() + stripplot()

(a) Critique of the Original Chart [5 Marks]

- Boxplots summarize with only five numbers (min, Q1, median, Q3, max) — they cannot show multimodality; a bimodal group and a genuinely unimodal group with the same median and quartiles will draw an identical-looking box.
- The 'ordinary-looking' box for Group C hid the fact that its 6 branches actually split into two distinct performance clusters (roughly 30L and 70L) rather than clustering around one typical value — a critical difference for regional strategy.
- This matters for decision-making: if the Regional Sales Director assumes Group C branches are all 'moderately performing' based on the box, they might apply a uniform intervention across all 6 branches, when in reality half need a completely different strategy than the other half.
- Boxplots also hide the actual sample size and density of points — a group with 6 branches and a group with 60 branches would produce similarly-shaped boxes, masking how much evidence backs the summary.
- The whiskers and IQR box further imply a single 'typical spread' around the median, which is actively misleading for Group C: there is no branch anywhere near its own median value of 49.1L, so the box is describing a value that does not represent any real branch.
- Because a boxplot's visual footprint doesn't change with the underlying shape, a reader scanning several boxplots side-by-side has no visual cue telling them to look more closely at Group C — the discovery in this scenario only happened 'by accident' while inspecting raw data, which should not be the primary safeguard against this kind of error.

Quantitative confirmation of the critique

Descriptive statistics per group, including a bimodality coefficient (BC), where BC > 0.555 is a common heuristic flag for possible bimodality in larger samples:

Group	Mean	Median	Std Dev	Min	Max	Bimodality Coeff.
A	40.6	40.1	3.5	36.8	46.8	0.297
B	54.2	54.6	4.9	46.2	60.1	0.225
C	49.7	49.1	21.7	29.5	71.1	0.234
D	50.2	49.3	7.9	38.8	60.2	0.195

Notice Group C's standard deviation (21.7) is roughly 3-4x the other groups' — a strong numeric hint that something structurally different is happening, even though its mean and median look unremarkable next to Groups B and D. Interestingly, the bimodality coefficient itself stays low for all groups here: with only n = 6 branches per group, the small-sample correction term in the BC formula dominates and washes out the signal — a limitation discussed further in part (d). This is precisely why the visual redesign below, not a single summary statistic, is the more trustworthy diagnostic at this sample size.

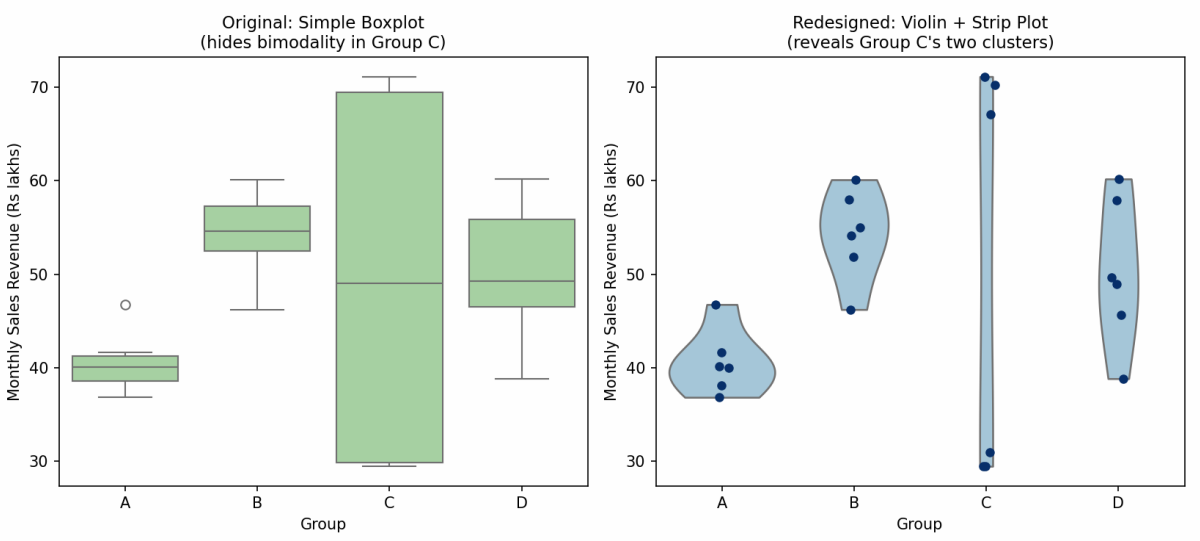
(b) Justification for a Bean/Violin+Strip Plot [5 Marks]

A bean plot (or violin + strip-plot combination) shows the full estimated density shape of each group's monthly sales revenue, not just five summary numbers, while a strip plot overlays every individual branch's actual revenue value as a point. Because there are only 6 branches per group, plotting every raw point is entirely feasible and highly informative — it directly exposes whether a group's revenue values cluster into one hump or two, exactly the bimodality that boxplots erased in this scenario.

This combination gives the Regional Sales Director both the shape (density) and the exact branch-level detail (strip points) needed for a review, without requiring a large sample size to be trustworthy — unlike the numeric bimodality coefficient above, a strip plot shows the raw truth of the data directly, so with only 6 branches per group it is actually the more reliable diagnostic of the two. Monthly sales revenue also naturally varies by branch size, local footfall, and regional demand; a chart that preserves every branch's individual value (rather than compressing to five numbers) lets the Director trace a concerning pattern back to specific, named branches for direct follow-up.

(c) Redesigned Chart [10 Marks]

Simulated dataset: monthly sales revenue (lakhs), 4 groups, 6 branch-level observations each. Group A ~ Normal(mean=40, sd=4); Group B ~ Normal(mean=55, sd=5); Group D ~ Normal(mean=48, sd=6); Group C deliberately bimodal: 3 branches ~ Normal(30, 2) and 3 branches ~ Normal(70, 2) so its overall mean/median look 'ordinary' near 50 despite no branch actually being near 50.



Left: original boxplot (Group C looks unremarkable). Right: redesigned violin+strip plot (Group C's two clusters are clearly visible).

Step 1 — simulate the data and compute group-level diagnostics

```
import numpy as np
import pandas as pd
from scipy import stats

np.random.seed(7)
group_A = np.random.normal(40, 4, 6)
group_B = np.random.normal(55, 5, 6)
group_C = np.concatenate([np.random.normal(30, 2, 3), np.random.normal(70, 2, 3)])
# bimodal
group_D = np.random.normal(48, 6, 6)

data = pd.DataFrame({
    "Revenue": np.concatenate([group_A, group_B, group_C, group_D]),
    "Group": ["A"]*6 + ["B"]*6 + ["C"]*6 + ["D"]*6
})

def bimodality_coefficient(x):
    n = len(x)
    g = stats.skew(x)
    k = stats.kurtosis(x, fisher=True)        # excess kurtosis
    correction = (3 * (n - 1)**2) / ((n - 2) * (n - 3))
    return (g**2 + 1) / (k + correction)

for g, vals in data.groupby("Group")["Revenue"]:
    print(g, "BC =", round(bimodality_coefficient(vals.values), 3),
          "Std =", round(vals.std(), 1))
```

Step 2 — build the redesigned violin + strip plot

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(11, 5))

sns.boxplot(data=data, x="Group", y="Revenue", ax=axes[0])
axes[0].set_title("Original: Simple Boxplot")

sns.violinplot(data=data, x="Group", y="Revenue", ax=axes[1], inner=None, cut=0)
sns.stripplot(data=data, x="Group", y="Revenue", ax=axes[1], color="black",
size=6, jitter=0.08)
axes[1].set_title("Redesigned: Violin + Strip Plot")

plt.tight_layout()
plt.savefig("q2_beanplot.png", dpi=150)
```

Insight the original chart did not reveal: Group C's violin shape clearly shows two separate humps (around 30L and 70L) with a gap in between, and the strip points confirm no branch actually sits near the group's own median (~ 49L). The boxplot's median/IQR summary was a statistical mirage — the redesigned chart shows Group C is really two different kinds of branches, not one moderately performing group, and because every branch is plotted as an individual point, the Director can immediately see this is a 3-vs-3 split rather than, say, one outlier branch dragging up an average.

(d) Reflection [5 Marks]

Remaining limitation #1 — bandwidth sensitivity: with only 6 data points per group, the estimated density (violin shape) is quite sensitive to the smoothing bandwidth chosen — a different bandwidth setting could make the bimodal shape look more or less pronounced than it really is, or even smooth it away entirely.

Remaining limitation #2 — small-sample numeric diagnostics are unreliable: as shown above, the formal bimodality coefficient failed to flag Group C at n = 6, even though the visual pattern is unambiguous. This means the redesigned chart must be read visually and cannot yet be backed by a single trustworthy summary number at this sample size.

How I would explain this to the Regional Sales Director (non-technical): “This new chart shows every branch's actual number as a dot, plus a shape showing how the revenues are distributed. For Group C, you can literally see two separate clumps of branches instead of one typical value — that's the pattern the old chart's single box was hiding. Because we only have 6 branches per group, the exact 'shape' drawn is a smoothed estimate, so I'd treat the two-cluster finding as a clear real signal — it's visible directly in the raw dots, not just a statistical artifact — but I would not over-interpret the finer wiggles of the curve itself, and I'd recommend we track a few more months of data to confirm the split persists.”

Question 3 — Visualizing Proportions: Stacked Bars & Stacked Densities (25 Marks)

Topic: Visualizing Proportions — Stacked Bars and Stacked Densities

Scenario: To show the composition of monthly sales revenue (lakhs) across 6 store branches, broken down into 11 sub-categories, a single stacked bar chart was built. The middle segments (mid-sized sub-categories) had almost no visible baseline to compare against, so the report's readers could not tell whether those segments had grown or shrunk compared to last year.

Tool(s) specified for redesign: Python pandas.DataFrame.plot(kind='bar', stacked=True) / seaborn.kdeplot(multiple='stack') | R ggplot2 geom_bar(position='stack') / geom_density(position='stack')

(a) Critique of the Original Chart [5 Marks]

- No common baseline for middle segments: every segment except the bottom one starts wherever the previous segment ended, so middle sub-categories have a different, shifting baseline in every bar — making it nearly impossible to visually compare their heights across branches or over time.
- 11 sub-categories is too many distinct colours for one stacked bar; several segments end up visually similar in colour and get compressed into slivers only a few pixels tall, especially for smaller sub-categories.
- Growth/shrinkage of any single mid-sized category is unreadable: because both the top and bottom edge of a middle segment move between bars, a segment can appear to 'shift' even if its actual value didn't change much, and vice versa — this is exactly the failure the Regional Sales Director flagged.
- The chart also collapses the underlying continuous revenue values into a single static composition per branch, giving no sense of how spread out or concentrated a sub-category's revenue distribution really is.
- There is no explicit year-over-year comparison built into the chart at all — even if a reader could perfectly judge each segment's height, the original chart only ever shows one time period, so 'grew or shrunk vs last year' cannot be answered by this chart under any circumstances, no matter how carefully it is read.

(b) Justification for the Redesign [5 Marks]

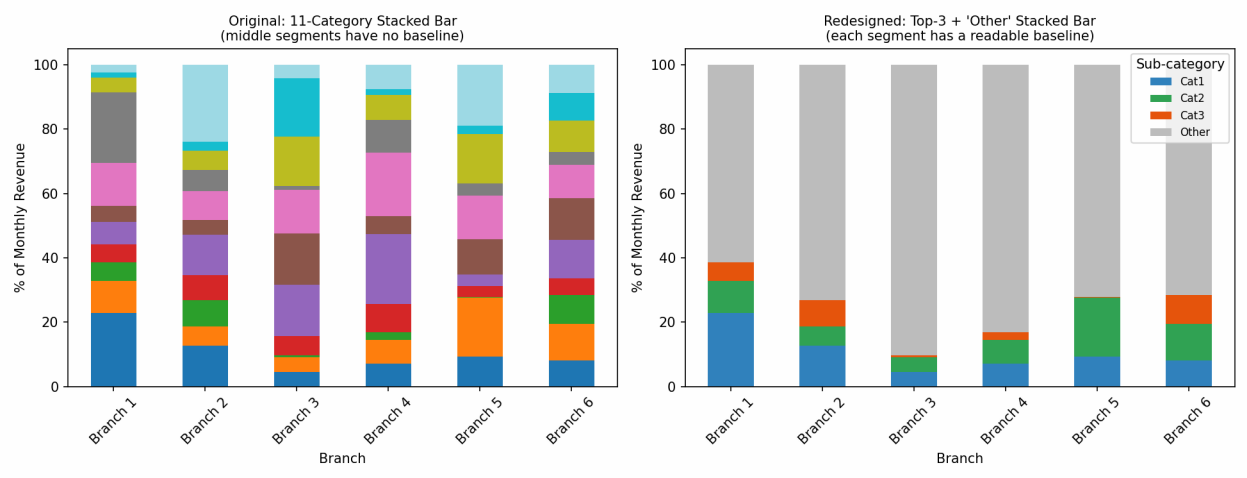
Reducing to a small number of well-chosen sub-categories (the three individually largest categories plus a single 'Other' bucket for the remaining eight) keeps every segment large enough to have a genuinely readable baseline, while still preserving the big-picture composition.

Critically, this redesign also directly fixes the scenario's stated complaint by adding an explicit this-year-vs-last-year comparison for the same four buckets, rather than relying on a reader's eye to compare two separate stacked-bar snapshots. Pairing this with a supporting stacked

density plot adds a third, complementary view: instead of just a composition snapshot per branch, it shows how revenue values within each category are actually distributed, which is valuable when 6 branches are being compared for similar underlying patterns rather than just totals. Together these views answer the Regional Sales Director's real question — which categories are moving, by how much, and compared to what baseline.

(c) Redesigned Chart [10 Marks]

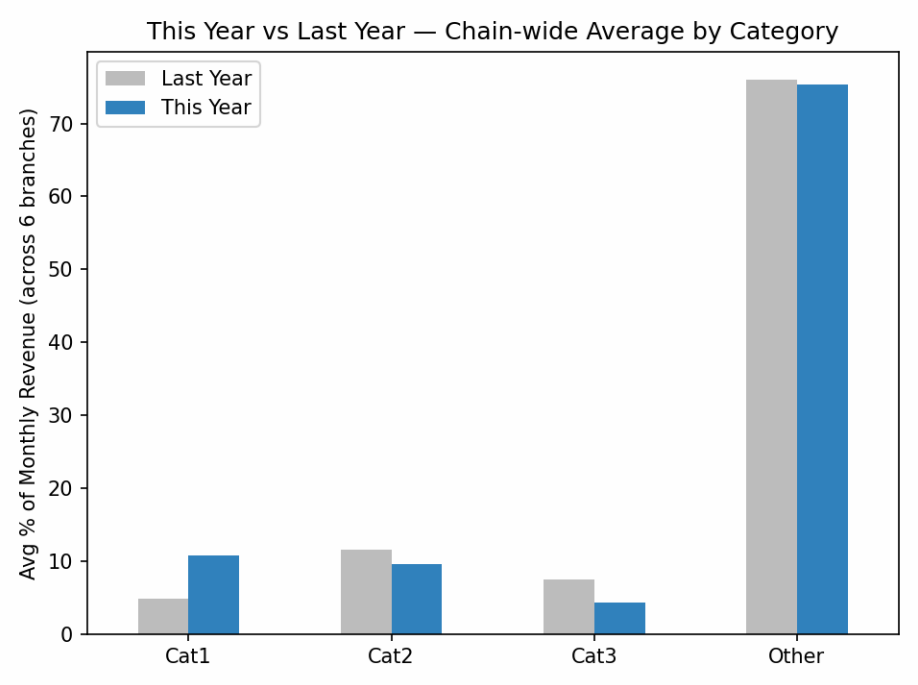
Simulated dataset: for each of the 6 branches, revenue composition across 11 sub-categories was drawn from a Dirichlet distribution (concentration = 2) to sum to 100%, once for 'this year' and once (independently redrawn) for 'last year.' For the redesign, the three largest sub-categories (Cat1, Cat2, Cat3) are kept individually and the remaining eight are combined into 'Other.'



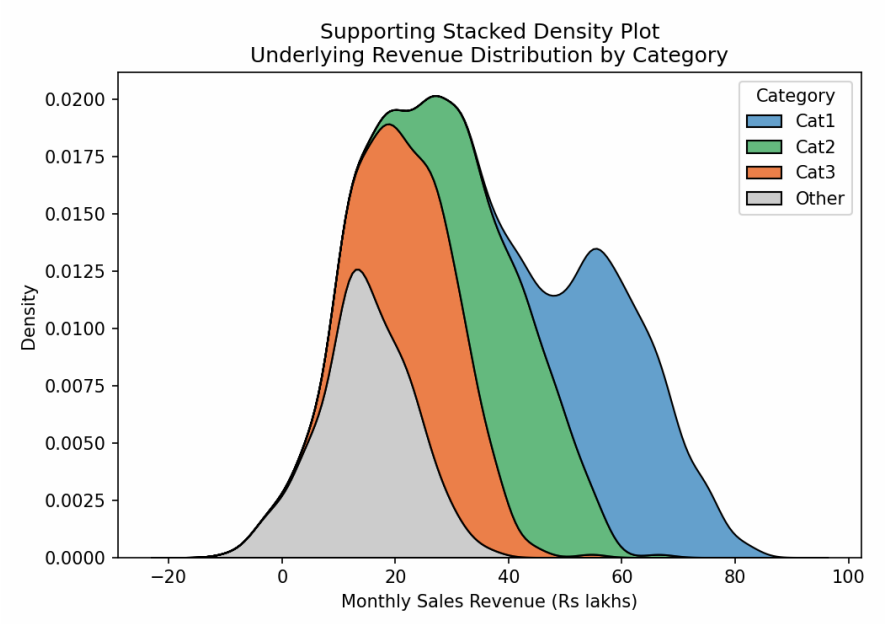
Left: original 11-category stacked bar (unreadable middle segments). Right: redesigned Top-3 + 'Other' stacked bar.

Chain-wide average composition, this year vs last year (% of revenue)

Category	Last Year	This Year	YoY Change (pp)
Cat1	4.8%	10.75%	+5.95
Cat2	11.6%	9.6%	-2.05
Cat3	7.5%	4.4%	-3.18
Other	76.0%	75.3%	-0.72



New supporting chart: chain-wide average This-Year vs Last-Year comparison — this is what directly answers 'did it grow or shrink?'



Supporting stacked density plot of the underlying continuous revenue pattern by category.

Step 1 — simulate this-year and last-year composition data

```
import numpy as np
import pandas as pd
```

```
np.random.seed(3)
branches = [f"Branch {i+1}" for i in range(6)]
subcats = [f"Cat{i+1}" for i in range(11)]

this_year = pd.DataFrame(np.random.dirichlet(np.ones(11)*2, size=6) * 100,
                         columns=subcats, index=branches)
last_year = pd.DataFrame(np.random.dirichlet(np.ones(11)*2, size=6) * 100,
                         columns=subcats, index=branches)

def bucket(df):
    out = df[["Cat1", "Cat2", "Cat3"]].copy()
    out["Other"] = df.drop(columns=["Cat1", "Cat2", "Cat3"]).sum(axis=1)
    return out

redesigned_this_year = bucket(this_year)
redesigned_last_year = bucket(last_year)
yoy_change = (redesigned_this_year.mean() - redesigned_last_year.mean()).round(2)
print(yoy_change)
```

Step 2 — plot the redesigned stacked bar, YoY comparison, and stacked density

```
import matplotlib.pyplot as plt
import seaborn as sns

# Redesigned Top-3 + Other stacked bar
redesigned_this_year.plot(kind="bar", stacked=True,
                          color=["#3182bd", "#31a354", "#e6550d", "#bdbdbd"])
plt.ylabel("% of Monthly Revenue"); plt.title("Top-3 + Other Stacked Bar by
Branch")
plt.savefig("q3_stacked_bar.png", dpi=150)

# NEW: This Year vs Last Year comparison (answers the "grew or shrunk?" question)
compare_df = pd.DataFrame({"Last Year": redesigned_last_year.mean(),
                           "This Year": redesigned_this_year.mean()})
compare_df.plot(kind="bar", color=["#bdbdbd", "#3182bd"])
plt.ylabel("Avg % of Monthly Revenue"); plt.title("This Year vs Last Year by
Category")
plt.savefig("q3_yoy_comparison.png", dpi=150)

# Supporting stacked density plot
sns.kdeplot(data=density_df, x="Revenue", hue="Category", multiple="stack")
plt.title("Stacked Density Plot of Revenue by Category")
plt.savefig("q3_stacked_density.png", dpi=150)
```

Insight the original chart did not reveal: with only 4 readable bands per branch, it becomes clear that 'Cat1' more than doubled its chain-wide average share (4.8% → 10.75%, +5.95 percentage points) while 'Cat3' shrank by roughly a third (7.5% → 4.4%, -3.18 percentage points) — a pattern that was completely invisible when it was split across 8 thin slivers in the original 11-category chart with no year-over-year reference at all. The stacked density plot further shows

'Cat3' has a noticeably tighter, lower-value spread than the other categories, consistent with its shrinking chain-wide share.

(d) Reflection [5 Marks]

Remaining limitation #1 — loss of sub-category detail: collapsing eight sub-categories into a single 'Other' bucket sacrifices detail — if two of those hidden sub-categories are moving in opposite directions (one growing, one shrinking), that offsetting movement is now invisible again, just at a coarser level than before.

Remaining limitation #2 — branch-level averaging hides variation: the YoY comparison chart is shown as a chain-wide average across all 6 branches; an individual branch could be moving in the opposite direction from the chain-wide trend, and that would be masked by averaging unless the reader also checks the branch-level stacked bars.

How I would explain this to the Regional Sales Director (non-technical): “We've kept the three categories that matter most on their own so you can track them clearly branch by branch, grouped the rest into one 'Other' bucket so the chart stays readable, and added a direct before-and-after comparison so you don't have to guess whether something grew or shrank. The trade-off is twofold: if something important is happening inside 'Other,' this chart won't show it, and the year-over-year comparison is a chain-wide average, so I'd recommend we drill into 'Other' and into any single branch that looks unusual separately.”

Question 4 — Visualizing Proportions: Parallel Sets (25 Marks)

Topic: Visualizing Proportions — Parallel Sets

Scenario: To understand how records move across three categorical dimensions relevant to 6 store branches (category, status, and final outcome), three separate grouped bar charts — one per dimension — were produced, asking the Regional Sales Director to mentally combine them to find common combinations.

Tool(s) specified for redesign: Python plotly.graph_objects.Parcats() | R ggforce::geom_parallel_sets() or ggalluvial

(a) Critique of the Original Chart [5 Marks]

- Three single-dimension bar charts each show only marginal totals (e.g., how many records are 'Electronics' in total) — none of them show how dimensions relate to each other, so co-occurring combinations (e.g., 'Electronics + Online + Returned') cannot be read off any single chart.
- Mentally combining three separate charts requires the reader to hold three sets of proportions in working memory simultaneously and multiply/cross-reference them by hand — this is cognitively demanding and error-prone, especially with three categories in one dimension and three in another.
- The charts give no sense of flow or relationship strength between specific category values across dimensions — e.g., whether 'Returned' outcomes are disproportionately linked to 'Online' orders cannot be seen at all, only inferred by guesswork.
- Splitting one dataset into three charts also triples the visual real-estate and reading effort needed for what is fundamentally one question: 'which combinations of category, status, and outcome occur together most often?'
- Because each bar chart is built from marginal totals only, it is mathematically impossible to recover the true joint distribution from the three charts alone — even a careful reader doing the mental arithmetic correctly would be forced to assume independence between dimensions, which the data below shows is false.

Quantitative confirmation of the critique

A chi-square test of independence between Status and Outcome on the simulated data confirms the dimensions are NOT independent — meaning any reader who mentally combined the three separate bar charts assuming independence would draw the wrong conclusion:

Status	Completed	Returned	Cancelled
In-Store	114 (84.4%)	17 (12.6%)	4 (3.0%)
Online	111 (67.3%)	37 (22.4%)	17 (10.3%)

Chi-square = 12.62, degrees of freedom = 2, p-value = 0.00182 → statistically significant association between Status and Outcome (reject independence at $\alpha = 0.05$). Online orders are

returned or cancelled roughly twice as often as In-Store orders — a relationship completely invisible in three separate single-dimension bar charts.

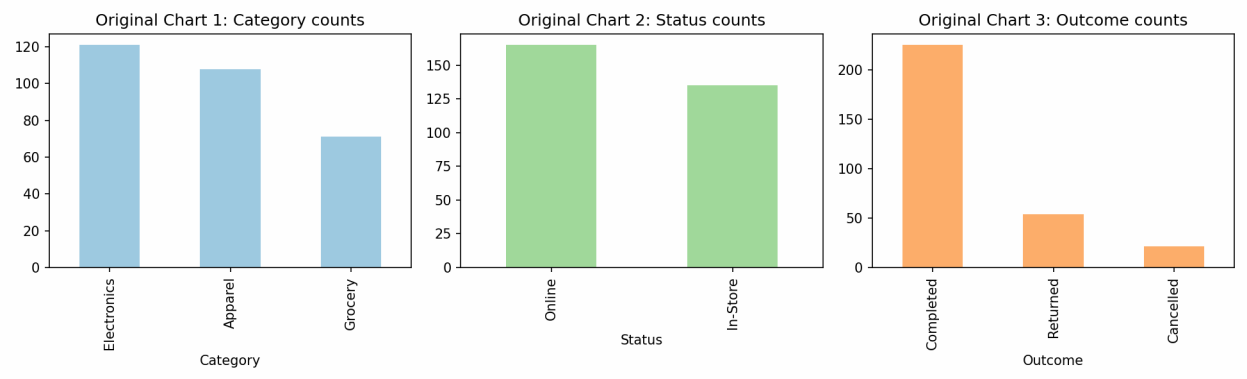
(b) Justification for a Parallel Sets Diagram [5 Marks]

A parallel sets (parallel categories) diagram places all three dimensions — Category, Status, and Outcome — as parallel axes and draws ribbons connecting each record's actual combination of values across all three, with ribbon thickness proportional to how many records share that path. This directly answers the question the three separate bar charts could not: which specific combinations across all three dimensions are common versus rare, and it does so without requiring the reader to assume independence between dimensions the way mentally combining three marginal bar charts does.

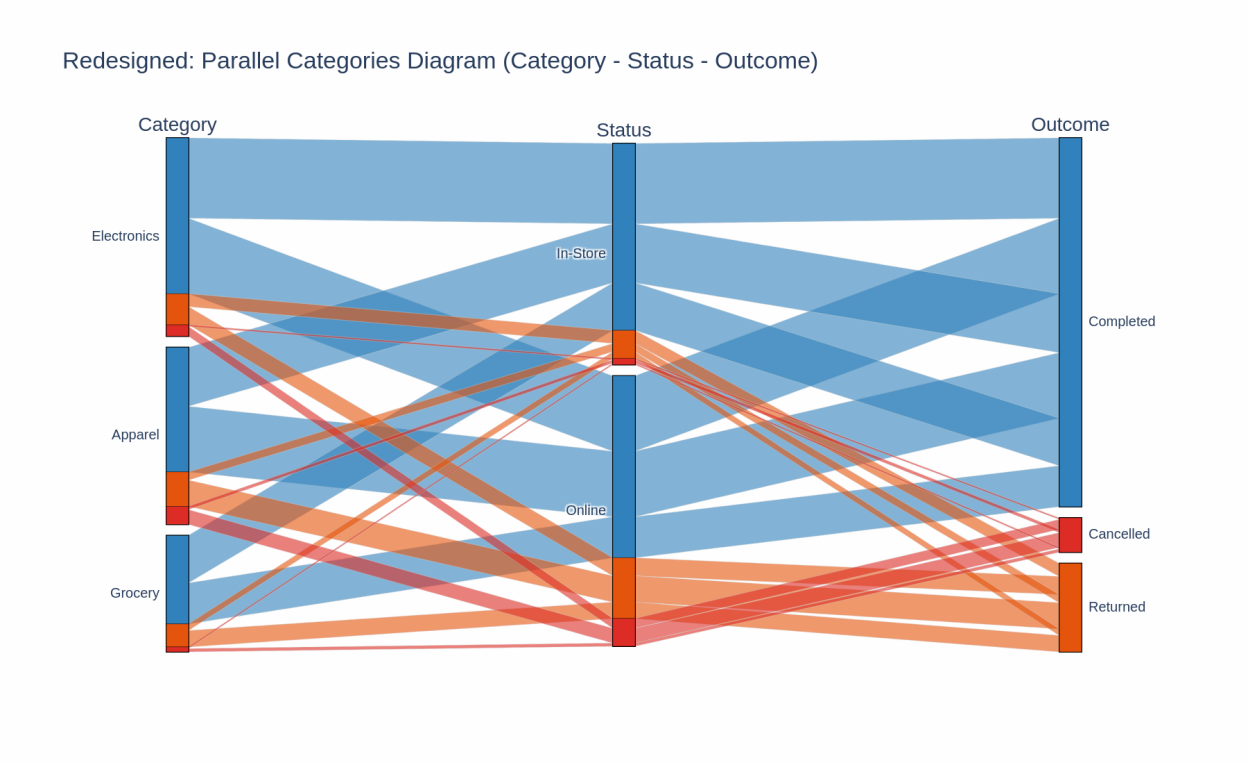
For a retail chain with sales spread across 6 store branches, this makes it immediate to spot that returns and cancellations cluster more heavily among online orders — confirmed statistically above — which no amount of mental cross-referencing of separate bar charts could reveal as reliably or as quickly, and which is exactly the kind of actionable, combination-level insight a Regional Sales Director needs to target a specific fix (e.g., reviewing the online checkout/delivery experience) rather than a generic, chain-wide one.

(c) Redesigned Chart [10 Marks]

Simulated dataset: 300 records with Category in {Electronics, Apparel, Grocery} (probabilities 0.40/0.35/0.25) and Status in {Online, In-Store} (0.55/0.45), sampled independently. Outcome was deliberately simulated to DEPEND on Status — Online orders: {Completed 0.65, Returned 0.22, Cancelled 0.13}; In-Store orders: {Completed 0.88, Returned 0.08, Cancelled 0.04} — to mimic a realistic scenario where a real cross-dimensional relationship exists for the redesign to reveal.



Original: three separate single-dimension bar charts (Category, Status, Outcome) shown side by side.



Redesigned: Parallel Categories diagram spanning Category → Status → Outcome, ribbons coloured by Outcome.

Step 1 — simulate correlated data and confirm the relationship statistically

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

np.random.seed(11)
n = 300
category = np.random.choice(["Electronics", "Apparel", "Grocery"], n, p=[0.40,
0.35, 0.25])
status = np.random.choice(["Online", "In-Store"], n, p=[0.55, 0.45])

outcome = []
for s in status:
    if s == "Online":
        outcome.append(np.random.choice(["Completed", "Returned", "Cancelled"],
p=[0.65, 0.22, 0.13]))
    else:
        outcome.append(np.random.choice(["Completed", "Returned", "Cancelled"],
p=[0.88, 0.08, 0.04]))

df = pd.DataFrame({"Category": category, "Status": status, "Outcome": outcome})

crosstab = pd.crosstab(df["Status"], df["Outcome"])
chi2, p, dof, expected = chi2_contingency(crosstab)
```

```
print(crosstab)
print(f"chi2={chi2:.2f}, dof={dof}, p={p:.5f}")   # p = 0.00182 -> significant
association

top_combos = df.groupby(["Category", "Status", "Outcome"]).size() \
               .sort_values(ascending=False).head(6)
print(top_combos)
```

Step 2 — build the redesigned Parallel Categories diagram

```
import plotly.graph_objects as go

dims = [
    dict(values=df["Category"], label="Category"),
    dict(values=df["Status"],   label="Status"),
    dict(values=df["Outcome"],  label="Outcome"),
]
color = df["Outcome"].map({"Completed": 0, "Returned": 1, "Cancelled": 2})

fig = go.Figure(go.Parcats(
    dimensions=dims,
    line=dict(color=color, colorscale=[[0, "#3182bd"], [0.5, "#e6550d"], [1,
"#de2d26"]]),
    hoveron="color",
))
fig.update_layout(title="Parallel Categories: Category - Status - Outcome")
fig.write_image("q4_parcats.png", scale=2)
```

Insight the original chart did not reveal: the diagram makes it visible that the 'Returned' and 'Cancelled' ribbons (orange/red) are noticeably thicker on the path passing through 'Online' than through 'In-Store' — consistent with the chi-square result above (p = 0.00182) — meaning problem outcomes are disproportionately associated with online orders, a cross-dimensional relationship that three separate single-dimension bar charts, each blind to the other two dimensions, had no way of showing. The top co-occurring combinations table further shows 'Electronics + In-Store + Completed' (49 records) as the single most common path, information no marginal bar chart set could produce.

(d) Reflection [5 Marks]

Remaining limitation #1 — scalability: with three dimensions of 3, 2, and 3 categories respectively, the diagram already has many crossing ribbons; if a fourth dimension or many more categories were added, the ribbons would overlap so much that individual paths become hard to trace visually, limiting how far this technique scales.

Remaining limitation #2 — association is not causation: the chi-square test confirms Status and Outcome are statistically associated, but neither the test nor the diagram explains why — e.g., whether online returns are driven by sizing/fit issues, delivery damage, or a different customer segment shopping online — that requires further investigation beyond what any single visualization can show.

How I would explain this to the Regional Sales Director (non-technical): “Instead of three separate charts you'd have to mentally stitch together, this one chart shows the actual paths records take — for example, you can now see, and we've confirmed it's not just chance, that online orders are more likely to end up returned or cancelled than in-store ones. The one caution is that this style of chart works best with a handful of categories in each dimension; if we start tracking many more categories, we'd need to simplify or split the view again to keep it readable, and we'd still need a follow-up investigation to understand exactly why online orders behave differently before deciding on a fix.”